PERSONAL FINANCE & BUDGET TRACKER

1. Project Scenario

Imagine a college student named Aarav. Every month he receives pocket money and also earns a little from a part-time tutoring job. But by the end of the month, he never knows where his money went — he just knows it's gone. He tries writing expenses in a notebook, but he keeps losing track, forgets to add entries, and can never tell if he is spending too much on food, travel, or entertainment.

Aarav decides to solve this problem himself since he is learning Python. He wants to build a simple desktop application where he can:

- Log in with his own account so his data stays private.
- Add his income (pocket money, tutoring fees, gifts, etc.).
- Add his expenses under categories like Food, Travel, Rent, Entertainment, and Bills.
- Set a monthly budget for each category, e.g. ₹2000 for Food.
- See instantly if he has crossed his budget in any category.
- View simple charts/reports showing where his money goes.
- Look back at any previous month's spending history.

This project walks you through building exactly that application — a real, usable tool — while learning GUI development with CustomTkinter and database handling with SQLite3. It is intentionally scoped for a beginner: no web servers, no cloud accounts, no complicated frameworks. Everything runs locally on one computer, in one Python project.

1.1 What You Will Learn

- Building multi-page desktop GUIs with CustomTkinter (frames, widgets, navigation).
- Structuring a Python project into clean, reusable modules instead of one giant file.
- Designing a relational database schema with SQLite3 (tables, keys, relationships).
- Performing CRUD operations (Create, Read, Update, Delete) from a GUI.
- Basic authentication (username/password login) stored securely with hashing.
- Simple data visualization (bar/pie charts) using matplotlib inside CustomTkinter.
- Packaging a Python desktop app so it can be run like a real program.

2. Project Objectives

1. Allow a user to register and log in securely.
2. Let the user record income and expense transactions with a category, amount, date, and note.
3. Let the user create and manage custom categories.

4. Let the user set a monthly budget limit per category and warn them when they overspend.
5. Show a dashboard summarizing total income, total expenses, and remaining balance.
6. Show visual reports (pie chart by category, bar chart by month).
7. Store all data locally in an SQLite3 database file so nothing is lost when the app closes.
8. Keep the code organized so a beginner can navigate and extend it easily.

3. Tech Stack & Prerequisites

3.1 Tools & Libraries

Tool / Library	Purpose	Install Command
Python 3.10+	Core programming language	Download from python.org
customtkinter	Modern-looking GUI widgets	pip install customtkinter
sqlite3	Local database (built into Python)	No install needed
bcrypt	Securely hash user passwords	pip install bcrypt
matplotlib	Charts for reports page	pip install matplotlib
Pillow (PIL)	Icons / images in the UI	pip install pillow
tkcalendar	Date-picker widget for transactions	pip install tkcalendar

3.2 Prerequisite Knowledge

- Basic Python: variables, functions, classes, loops, if/else.
- Basic idea of what a database table and a row/column is (helpful but not required — explained in Section 6).

No prior CustomTkinter or SQL experience is required — this guide explains each piece as it is introduced.

4. Folder Structure

Keeping files organized from day one makes the project much easier to extend later. Use the structure below:

```
finance_tracker/
│
├── main.py                     # App entry point — starts the application
│
├── database/
│   ├── db_manager.py           # All SQLite connection + query functions
│   └── finance_tracker.db      # SQLite database file (auto-created on first run)
│
├── gui/
│   ├── app.py                  # Main App window, controls page switching
│   ├── login_page.py           # Login screen
│   ├── register_page.py        # Sign-up screen
│   ├── dashboard_page.py       # Home/summary screen after login
│   ├── add_transaction_page.py # Form to add income/expense
│   ├── transactions_page.py    # Table/list of all transactions
│   ├── categories_page.py      # Manage categories
│   ├── budget_page.py          # Set & view monthly budgets
│   ├── reports_page.py         # Charts and analytics
│   └── settings_page.py        # Profile, password change, theme
│
├── models/
│   ├── user.py                 # User data class + auth helper functions
│   ├── transaction.py          # Transaction data class
│   ├── category.py             # Category data class
│   └── budget.py               # Budget data class
│
├── utils/
│   ├── validators.py           # Input validation (amount, date, empty fields)
│   ├── security.py             # Password hashing / verification
│   └── constants.py            # Colors, fonts, fixed category list, paths
│
├── assets/
│   ├── icons/                  # PNG/SVG icons used in the sidebar & buttons
│   └── logo.png
│
├── requirements.txt            # List of pip packages the project needs
└── README.md                   # How to install & run the project
```

Beginner tip: You do not have to create every file on day one. Start with main.py, database/db_manager.py, and gui/login_page.py, get those working, then add one page at a time.

5. What Each File Does

5.1 Root & Database Layer

Page / File	Purpose	Key Functions
main.py	Entry point of the whole app. Creates the database if it doesn't exist, then launches the login window.	• Call db_manager.initialize_db() • Create and run the App() class from gui/app.py

Page / File	Purpose	Key Functions
database/db_manager.py	The only file that talks directly to SQLite. Every other file asks this module to fetch or save data — this keeps SQL code in one place.	• get_connection() • initialize_db() • insert_user() / verify_user() • insert_transaction() / get_transactions() • insert_category() / get_categories() • set_budget() / get_budget_status()
models/user.py, transaction.py, category.py, budget.py	Simple Python classes that represent one row of each table, so the rest of the code works with objects instead of raw tuples.	• __init__() to store the fields • to_dict() for easy display in tables
utils/security.py	Handles password hashing so raw passwords are never stored in the database.	• hash_password() • check_password()
utils/validators.py	Checks user input before it reaches the database (e.g. amount must be a positive number).	• is_valid_amount() • is_valid_date() • is_not_empty()
utils/constants.py	Central place for colors, fonts, default categories, and the database file path so they aren't repeated everywhere.	• COLORS, FONTS, DEFAULT_CATEGORIES, DB_PATH

5.2 GUI Pages

Page / File	Purpose	Key Functions
gui/app.py	The main container window. Holds a sidebar and swaps the visible page (frame) when the user clicks a menu button.	• show_frame(page_name) • build_sidebar() • logout()
gui/login_page.py	First screen the user sees. Lets an existing user sign in.	• on_login_click() — verifies credentials • go_to_register() — switch to sign-up
gui/register_page.py	Lets a brand-new user create an account.	• on_register_click() — validates + saves new user • back_to_login()
gui/dashboard_page.py	Home screen after login. Shows a quick snapshot of finances.	• load_summary() — total income, expense, balance • show_recent_transactions() • show_budget_alerts()
gui/add_transaction_page.py	A form to add a new income or expense entry.	• on_save_click() — validate + insert into DB • clear_form()
gui/transactions_page.py	Displays every transaction in a scrollable, searchable/filterable table.	• load_transactions() • filter_by_category() / filter_by_date() • delete_transaction() / edit_transaction()
gui/categories_page.py	Lets the user view, add, rename, or delete spending categories.	• add_category() • edit_category() • delete_category()
gui/budget_page.py	Lets the user set a monthly limit per category and shows progress bars for each.	• set_budget() • load_budget_progress() — spent vs limit per category

Page / File	Purpose	Key Functions
gui/reports_page.py	Visual analytics: pie chart of spending by category, bar chart of spending by month.	• draw_pie_chart() • draw_monthly_bar_chart() • filter_by_month()
gui/settings_page.py	Lets the user update their name/email, change password, or switch light/dark theme.	• update_profile() • change_password() • toggle_theme()

6. Database Schema (SQLite3)

The database file finance_tracker.db will contain four tables. A quick primer if you're new to this: a table is like a spreadsheet, each row is one record, and a PRIMARY KEY uniquely identifies each row. A FOREIGN KEY links a row in one table to a row in another (e.g. a transaction 'belongs to' a user).

6.1 Entity Overview

- users — one row per registered person.
- categories — spending/income categories, owned by a user (e.g. Food, Travel, Salary).
- transactions — every income or expense entry, linked to a user and a category.
- budgets — the monthly limit a user sets for a category.

6.2 users Table

Column	Type	Notes
user_id	INTEGER	PRIMARY KEY AUTOINCREMENT
full_name	TEXT	NOT NULL
username	TEXT	NOT NULL, UNIQUE
password_hash	TEXT	NOT NULL — store hashed password only, never plain text
created_at	TEXT	DEFAULT CURRENT_TIMESTAMP

```
CREATE TABLE users (
    user_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name     TEXT NOT NULL,
    username      TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at    TEXT DEFAULT CURRENT_TIMESTAMP
);
```

6.3 categories Table

Column	Type	Notes
category_id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FOREIGN KEY → users(user_id)
name	TEXT	NOT NULL, e.g. 'Food', 'Travel', 'Salary'
type	TEXT	'income' or 'expense'

```
CREATE TABLE categories (
    category_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER NOT NULL,
    name        TEXT NOT NULL,
    type        TEXT CHECK(type IN ('income','expense')) NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

6.4 transactions Table

Column	Type	Notes
transaction_id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FOREIGN KEY → users(user_id)
category_id	INTEGER	FOREIGN KEY → categories(category_id)
amount	REAL	NOT NULL, always stored as a positive number
type	TEXT	'income' or 'expense'
note	TEXT	Optional description
date	TEXT	Format YYYY-MM-DD

```
CREATE TABLE transactions (
    transaction_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id        INTEGER NOT NULL,
    category_id    INTEGER NOT NULL,
    amount         REAL NOT NULL,
    type           TEXT CHECK(type IN ('income','expense')) NOT NULL,
    note           TEXT,
    date           TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

6.5 budgets Table

Column	Type	Notes
budget_id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FOREIGN KEY → users(user_id)
category_id	INTEGER	FOREIGN KEY → categories(category_id)
month	TEXT	Format YYYY-MM, e.g. '2026-07'
limit_amount	REAL	NOT NULL — the budget cap for that category & month

```
CREATE TABLE budgets (
    budget_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id      INTEGER NOT NULL,
    category_id  INTEGER NOT NULL,
    month        TEXT NOT NULL,
    limit_amount REAL NOT NULL,
```

```
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

6.6 How the Tables Relate

One user has many categories, many transactions, and many budgets. Each transaction belongs to exactly one category. Each budget row means: 'for this user, in this month, this category has this spending limit.' This is a simple one-to-many relationship model — you don't need anything more advanced than that for this project.

7. Application Flow (Page-by-Page)

Below is the order a user naturally moves through the app, and exactly what happens on each screen.

7.1 Login Page

- Two input fields: Username, Password.
- 'Login' button → checks credentials against the users table using the hashed password.
- On success → opens the Dashboard page.
- On failure → shows an inline error message ('Invalid username or password').
- A link/button 'Create an account' → opens the Register page.

7.2 Register Page

- Fields: Full Name, Username, Password, Confirm Password.
- Validates that username is not already taken and password matches confirmation.
- On success → hashes the password, inserts a new row into users, then returns to Login.
- Also creates a few default categories for the new user (Food, Travel, Bills, Salary, etc.).

7.3 Dashboard Page (Home)

- Shows three summary cards: Total Income, Total Expenses, Current Balance (for the selected month).
- Shows a short list of the 5 most recent transactions.
- Shows a warning banner if any category has exceeded its budget this month.
- Sidebar navigation to every other page is always visible from here.

7.4 Add Transaction Page

- A form with: Amount, Type (Income/Expense toggle), Category dropdown, Date picker, Note field.
- 'Save' button validates the input (amount > 0, date not empty) then inserts a row into transactions.
- Shows a success message and clears the form so another entry can be added quickly.

7.5 Transactions Page

- A scrollable table listing every transaction: date, category, type, amount, note.
- Filter dropdowns for category, type, and month.
- Each row has Edit and Delete buttons.
- Deleting asks for confirmation before removing the row from the database.

7.6 Categories Page

- Lists all categories the user has created, split into Income and Expense tabs.
- 'Add Category' button opens a small form (name + type).
- Rename and delete options for each category (deleting is blocked or warned if transactions use it).

7.7 Budget Page

- Lets the user pick a month and set a spending limit per category.
- Shows a progress bar per category: amount spent vs. limit, colored green/yellow/red.
- Automatically recalculates whenever a new transaction is added in Add Transaction Page.

7.8 Reports Page

- Month selector at the top.
- Pie chart: expense breakdown by category for the selected month.
- Bar chart: income vs. expense trend across the last 6 months.
- Built using matplotlib embedded inside the CustomTkinter frame.

7.9 Settings Page

- Update full name / username.
- Change password (asks for current password, then new password twice).
- Toggle between Light and Dark theme (CustomTkinter's set_appearance_mode).
- Logout button → returns to the Login page.

8. Suggested Build Order (Beginner Roadmap)

Don't try to build everything at once. Follow these phases in order — each phase gives you something that actually runs.

9. Phase 1 – Setup: Create the folder structure, install libraries, write requirements.txt.
10. Phase 2 – Database: Write db_manager.py, create all four tables, test with a temporary script that inserts and prints dummy rows.
11. Phase 3 – Authentication: Build Login and Register pages, wire them to the users table with password hashing.
12. Phase 4 – Core CRUD: Build Add Transaction and Transactions pages so a logged-in user can add, view, edit, and delete entries.
13. Phase 5 – Categories: Build the Categories page so income/expense categories aren't hard-coded.
14. Phase 6 – Dashboard: Build the summary cards and recent-transactions list, pulling live totals from the database.
15. Phase 7 – Budgets: Build the Budget page and the over-budget warning logic.
16. Phase 8 – Reports: Add matplotlib charts on the Reports page.
17. Phase 9 – Polish: Add Settings page, theme toggle, icons, input validation messages, and error handling everywhere.
18. Phase 10 – Packaging: Write the README, freeze requirements.txt, optionally bundle with PyInstaller into a single .exe.